

Plano de implementação — Experimento v3: o contador como fonte de estrutura

Oct 7, 2026 · @Nycolas

Contexto e objetivo

O experimento v3 mede se um observador passivo, que vê só criptogramas e nonces de contador, consegue subir o piso de rodadas de Ascon, GIFT-COFB, Grain-128AEAD e Schwaemm256-128 usando três coisas que o pipeline atual não usa: um detector sobre o conjunto em vez de acurácia por par, mais de duas mensagens por dispositivo (cubos sobre os bits baixos do contador) e diferenças de nonce em outras posições. Os pisos de referência estão na branch `piso-rodadas-v2`: Ascon 3/12, GIFT-COFB 3/40, Grain 28/256, Schwaemm 2/11, obtidos por `run_floor.py` + `report_floor.py` com XOR de pares consecutivos, 300 chaves × 100 pares, modelos sobre bits brutos, BH-FDR $q = 0,05$ e monotonicidade.

Diagnóstico que motiva o v3, em três frases. O detector atual é acurácia por par com ~6 mil pares de teste, cujo intervalo de confiança ($\pm 1,3$ pontos) é maior que o viés esperado na rodada piso+1 (décimos de ponto por bit), então "acaso" na tabela significa "não medível com este teste", não "ausente". O par consecutivo é a diferença de primeira ordem; 2^d mensagens consecutivas com contador alinhado formam um cubo de dimensão d sobre os d bits baixos do nonce, e a soma dos criptogramas num bit onde o XOR dos claros é determinístico (bit 7 do ASCII) é exatamente a soma do keystream sobre o cubo, um distinguidor de ordem superior que o contador dá de graça. No GIFT-COFB o nonce atravessa duas cifragens antes do primeiro byte observável, então "3 rodadas" ali são 6 rodadas de exposição; a comparação entre algoritmos precisa dessa coluna.

Hipóteses pré-registradas (o relatório final deve confrontar cada uma, confirmada ou não):

Hipótese	Algoritmo	Ganho esperado no piso	Sub-experimento
H1: viés por bit sobre o conjunto detecta a rodada piso+1	Ascon, Schwaemm	+1	A
H2: agregação por dispositivo leva o acerto no piso a >95% com <100 pares	todos	pares necessários por rodada	A

Hipótese	Algoritmo	Ganho esperado no piso	Sub-experimento
H3: cubos de dimensão 1–10 em bits limpos dão soma zero acima do piso	Grain	de 28 para >50 clocks	B
H4: idem	Ascon	+1 a +2	B
H5: diferença em posição $j > 0$ sobrevive a mais rodadas que $j = 0$	GIFT, Ascon	+1	C
H6: um bloco de AD reduz o piso do GIFT em ~1 rodada (uma cifragem a mais)	GIFT	-1 (confirma mecanismo)	D

Escopo fixado e suposições

Entra: um módulo novo `scripts/reduced_rounds/exp_contador/` com um gerador compartilhado e quatro sub-experimentos (A, B, C obrigatórios; D opcional), reutilizando as variantes de rodadas, o critério estatístico e o formato de relatório já existentes. Fica de fora: redes neurais grandes (ResNet/multi-par), mudança da fonte do Ascon (`ascon128av13` → AEAD128 do SP 800-232) e o teto com texto conhecido; são tarefas separadas e não devem entrar neste PR.

Decisões já tomadas, para o Claude Code não reabrir:

1. Modelo de ameaça inalterado: ciphertext-only, mesma chave por dispositivo, nonce de contador público, claros distintos e desconhecidos, texto natural (corpus Gutenberg já usado). Nada de claro conhecido ou escolhido.
2. Política de redução do Ascon e do Schwaemm: `ambos ( pa = r , pb = min(r, 8) ; big = r , slim = min(r, 7) )`, a mesma da Tabela 1, para que os pisos sejam comparáveis. A política `dados` não entra.
3. GIFT-COFB em granularidade de uma rodada via `PureGiftCOFB` (Python puro); para o sub-experimento B, que precisa de mais mensagens, limitar o GIFT a dispositivos com 256 mensagens e aceitar o custo (~1 ms por mensagem).
4. Unidade de amostra: o dispositivo (uma chave, um contador, N mensagens consecutivas), não o par. Toda métrica é reportada por dispositivo e por rodada.
5. "Bit limpo" = posição de bit onde o XOR (sub-exp. A e C) ou a soma módulo 2 (sub-exp. B) dos claros é determinística, estimada empiricamente no corpus e nunca assumida; o conjunto de bits limpos é parte do relatório.

6. Detecção = teste de hipótese com FDR (BH, $q = 0,05$) sobre todas as configurações do v3, mais monotonicidade e teto de controle, exatamente como `report_floor.py` já faz; a acurácia por par vira métrica secundária.
7. Mensagens de 64 bytes de claro, como no v2, porque o sinal está no primeiro bloco e na tag.
8. Contador começa em zero por padrão (cenário "recém-ligado"), com o braço `sorteado` apenas no sub-experimento A, para manter a comparação com a Tabela 1.

Suposição a validar no primeiro dia: que os `.npz` de pares do v2 (`pares_texto_k300.npz` etc.) podem ser lidos pelo novo detector do sub-experimento A sem regeneração. Se o formato não guardar o nonce de cada par, regenerar com o gerador novo.

Reaproveitamento do código existente

Tudo abaixo está na branch `piso-rodadas-v2`; o v3 importa, não copia. Antes de escrever código, ler os docstrings de cada arquivo da tabela, porque eles registram decisões (Regras de Ouro) que o v3 deve manter.

Módulo	O que o v3 usa dele	Não mexer em
<code>scripts/reduced_rounds/floor_algos.py</code>	<code>ALGOS[algo].cipher(r, politica="ambos")</code> para construir o cifrador de qualquer contagem; <code>key_bytes</code> , <code>nonce_bytes</code> , <code>tag_bytes</code> , <code>rounds_padrao</code> , <code>fracao_da_spec</code>	As políticas e os limites de rodada
<code>scripts/reduced_rounds/build_variant.py</code>	Build das variantes compiladas (Ascon, Schwaemm, Grain); cache em <code>build/reduced_rounds/<nome>/</code>	Fontes vendorizadas e o patch de rodadas
<code>scripts/reduced_rounds/gift_pure_cipher.py</code>	<code>PureGiftCOFB(rounds).encrypt(key, nonce, pt, ad)</code> com rodada única; já aceita <code>ad</code>	A verificação contra o binário em 40 rodadas
<code>scripts/reduced_rounds/reduced_wrapper.py</code>	<code>ReducedRoundsCipher.encrypt(key, nonce, plaintext, ad=...)</code> , mesma interface do GIFT puro	—
<code>scripts/reduced_rounds/run_floor.py</code>	Modelo de geração: um DRBG por fonte (chaves, nonces, textos) com <code>SEED_GEN</code> , <code>impressao_digital</code> + <code>conferir_cache</code> para caches <code>.npz</code> , braços <code>texto / aleatorio</code> , <code>_log</code> , contador zero / sorteado. Copiar o padrão de cache, não o código de pares	A lógica de pares do v2 (fica como está, é a baseline)
<code>scripts/reduced_rounds/run_pilot.py</code>	<code>_TextPlaintextSampler(corpora_dir, drbg)</code> e <code>CORPORA_DIR</code> para claros de texto	—
<code>src/crypto/ctr_drbg.py</code>	<code>CTRDRBG(seed, label)</code> com <code>generate(n)</code> , <code>randint</code> , <code>random_key(16)</code> ; cada fonte de aleatoriedade com seu <code>label</code>	—

Módulo	O que o v3 usa dele	Não mexer em
<code>scripts/reduced_rounds/report_floor.py</code> + <code>scripts/consolidate_v2.apply_bh_fdr</code>	<code>apply_bh_fdr(df, q)</code> , cálculo do teto de controle, regra de monotonicidade e impressão dos pisos. O v3 produz JSONL no mesmo esquema de colunas para passar pelo mesmo relatório, acrescentando colunas novas em vez de renomear	O critério em si
<code>scripts/reduced_rounds/ablação_região_pares.py</code>	O recorte por região (bloco 1 = bits 0-127, blocos 2-4, tag) para reportar onde o sinal mora	—
<code>src/eval/reporting.py</code> (<code>report_eval</code>) e <code>scripts/run_v2_caminho_a.py</code> (<code>build_models</code> , <code>get_proba</code>)	Modelos clássicos e relatório por fold, para a métrica secundária por par	—
<code>tests/test_floor_algos.py</code> , <code>tests/test_gift_pure_cipher.py</code> , <code>tests/test_calibração_nulo.py</code>	Padrão de teste: cada gerador e detector novo ganha teste no mesmo estilo	—

Três fatos do código atual que mudam o desenho do v3 e que o Claude Code deve confirmar lendo `run_floor.py` antes de começar:

1. Os pares do v2 são `(c, c+1)` com `c` par (`off & ~1`), então a diferença de nonce é sempre exatamente 1 bit na posição 0, inclusive no braço `sorteado`. A queda do GIFT de 3 para 2 com contador sorteado vem dos bits altos do nonce serem aleatórios (vizinhança da diferença), não de diferenças de vários bits. O sub-experimento C precisa fixar a vizinhança para isolar o efeito da posição.
2. O contador é big-endian sobre o nonce inteiro: os bits baixos ficam no último byte. No Ascon isso cai no bit 0 da palavra x4; no Grain, no último byte dos 12 do nonce; no Schwaemm, no último dos 32. A posição onde a diferença entra na permutação é, portanto, fixa e não a mais favorável; registrar isso no relatório.
3. Os dispositivos usam os mesmos nonces entre algoritmos (offsets compartilhados), e o contador `zero` zera todos os bits altos. Para cubos (sub-exp. B) o contador `zero` é o caso ideal: 2^d mensagens consecutivas a partir de 0 formam o cubo sobre os d bits baixos sem nenhum alinhamento extra.

Geração de dados

Um gerador só, `exp_contador/gerar.py`, produz dispositivos e grava criptogramas crus com seus nonces; cada sub-experimento deriva suas features na leitura. Isso evita regenerar quando a feature muda e permite auditar qualquer amostra.

Dispositivo. Uma chave de `CTRDRBG(label="v3-key")`, um contador inicial (`zero`, ou sorteado e alinhado a múltiplo de 2^D com `label="v3-nonceoffset"`), N mensagens consecutivas com claros de `_TextPlaintextSampler` (`DRBG label="v3-plaintext"`),

AD vazio salvo no sub-exp. D. O mesmo dispositivo (chave, offset, claros) é cifrado por todos os algoritmos e todas as contagens de rodada, como no v2, para que a comparação seja só algorítmica.

Tamanhos. $N = 100$ para o braço de pares (compatível com o v2); $N = 2^D$ com $D = 10$ (1024 mensagens) para cubos no Ascon, Schwaemm e Grain; $N = 256$ ($D = 8$) para o GIFT puro em Python. Número de dispositivos: 300 por configuração como baseline, 3 000 onde a família A precisar de poder (ver aceite). Custo de referência: GIFT puro $\approx 1,1$ ms por mensagem, então $300 \text{ dispositivos} \times 256 \text{ mensagens} \times 12 \text{ contagens de rodada} \approx 15 \text{ min}$; os três compilados são desprezíveis.

Braços. `texto` (claros Gutenberg) é o braço principal. `aleatorio` (claros uniformes) é o controle negativo obrigatório em toda configuração: ali o XOR e as somas de criptogramas são uniformes por construção, e qualquer detecção é falso positivo. A rodada de especificação (Ascon 12, GIFT 40, Grain 256, Schwaemm 11) entra em toda varredura como teto de controle.

Cache. Um `.npz` por (algoritmo, braço, N , contador, política), com `impressao_digital` e `conferir_cache` iguais aos de `run_floor.py`, chaveado por contagem de rodadas (`r{r}`), incremental. Conteúdo por contagem: `ct` com forma `(dispositivos, N, 64 + tag_bytes)` em `uint8`, `nonce` com forma `(dispositivos, N, nonce_bytes)`, e, só no braço `texto`, `pt_mask`: a máscara de bits limpos estimada para aquele lote (ver sub-exp. B). Nunca gravar os claros nem as chaves no cache de resultados; guardar as seeds no manifesto.

Bits limpos (estimativa, não suposição). Para o braço `texto`, com os claros ainda em memória, calcular por posição de bit do primeiro bloco a fração de pares consecutivos com XOR dos claros igual a zero (`p_xor[bit]`) e, para cada d , a fração de cubos com soma dos claros igual a zero (`p_cubo[d][bit]`). Gravar os dois vetores no manifesto. Bit limpo = `p >= 0,999`. Esperado para ASCII: bit 7 de cada byte limpo nos dois casos; bits 5 e 6 enviesados mas não limpos. Se o corpus tiver acentos ou chinês, os bytes multibyte mudam a máscara, e é por isso que ela é estimada por lote e reportada.

Varredura de rodadas. `rounds_padrao` de `floor_algos.py`, mais a rodada `piso+1` e `piso+2` de cada algoritmo em densidade total: Ascon 1–8 e 12; GIFT 1–12, 20, 40; Grain 1–12, 16–64 de 4 em 4, 96, 128, 160, 256; Schwaemm 1–11.

Sub-experimento A: detector por viés de bit e agregação por dispositivo

A pergunta é se a rodada `piso+1` de cada algoritmo tem viés mensurável quando o detector é o teste certo. Entrada: o XOR de pares consecutivos `(c, c+1)`, `c` par, do primeiro bloco mais a tag, como no v2; para cada dispositivo com N mensagens, $N/2$ pares

disjuntos.

Detector 1, viés por bit (primário). Para cada posição de bit b do XOR, contar sobre todos os pares de todos os dispositivos de teste quantas vezes o bit é 1; sob H_0 (uniforme) a contagem é Binomial($M, 1/2$). Estatística: $z_b = (k_b - M/2) / \sqrt{M/4}$. Teste global por configuração = qui-quadrado com um grau de liberdade por bit, somado só sobre os bits limpos ($p_{xor} \geq 0,999$) e, em separado, sobre todos os bits; reportar os dois. O p-valor global de cada (algoritmo, rodada, braço, região) entra no BH-FDR junto com as demais configurações do v3. Poder: com $M = 150\,000$ pares ($3\,000$ dispositivos $\times 50$), um viés de 0,5% por bit dá $z \approx 3,9$; é por isso que a família A pede $3\,000$ dispositivos na rodada piso+1 e piso+2.

Detector 2, razão de verossimilhança por dispositivo (agregação). Com os viéses $q_b = \Pr[\text{bit } b = 1]$ estimados no treino (dispositivos disjuntos), o escore de um dispositivo é a soma, sobre seus pares e bits, de $\log(q_b / 0,5)$ se o bit é 1 e $\log((1 - q_b) / 0,5)$ se é 0; é o detector de Bayes ingênuo, ótimo se os bits forem independentes, e o que a regressão logística aproxima. Classificar dispositivo vs. aleatório por limiar no escore; reportar acerto por dispositivo em função de k pares usados ($k = 1, 2, 5, 10, 20, 50$), e o k necessário para 95% em cada rodada. Esse k é a "complexidade de dados" do distinguidor passivo, comparável com a literatura.

Detector 3, modelos do v2 (secundário). Regressão logística e XGBoost sobre os bits, com o mesmo split por chave, para manter a comparação com a Tabela 1. Só entram na manchete se superarem o Detector 2 no mesmo dado; se não superarem, o relatório diz isso explicitamente.

Regiões. Bloco 1 (bits 0–127), blocos 2–4, tag, cada uma como configuração própria (reaproveitar o recorte de `ablaacao_regiao_pares.py`). Para o Ascon com `pa` reduzido, a tag vem de um P12 reduzido e pode carregar sinal; esse é um resultado a reportar, não a esconder.

Paridades (extensão de A, só se o Detector 1 der zero na rodada piso+1 do Ascon).

Acrescentar como features o XOR de todos os pares e trios de bits limpos do bloco 1 (com 16 bits limpos: 120 pares + 560 trios) e repetir os Detectores 1 e 2. Isso é o distinguidor diferencial-linear com ruído: o que sobrevive a 4–5 rodadas da inicialização do Ascon é a paridade de um subconjunto de bits (Dobraunig et al., CT-RSA 2015), que bits isolados não representam.

Contador. `zero` como principal e `sorteado` como réplica, para manter os dois cenários da Tabela 1.

Saída. Uma linha JSONL por (algoritmo, rodada, braço, contador, região, detector, k) com p-valor, estatística, M , acerto por dispositivo e IC bootstrap por dispositivo, no esquema que `report_floor.py` lê, mais um `.csv` com z_b por bit para a figura de importância por posição.

Sub-experimento B: cubos passivos a partir do contador

A pergunta é se somar 2^d criptogramas consecutivos, em vez de xorar dois, detecta mais rodadas. É o cube tester de Aumasson, Dinur, Meier e Shamir aplicado sem escolher nada: o contador fornece o cubo.

Construção. Num dispositivo com contador começando em múltiplo de 2^D e $N = 2^D$ mensagens, os índices $i = 0 \dots 2^d - 1$ formam um cubo de dimensão d sobre os d bits baixos do nonce, para todo $d \leq D$. A soma do cubo numa posição de bit b é $S_d(b) = \text{XOR}$ dos bits b dos 2^d criptogramas. Como $C_i = KS_i \oplus P_i$, vale $S_d(b) = (\text{soma do keystream}) \oplus (\text{soma dos claros})$; numa posição limpa para cubos (`p_cubo[d][b] >= 0,999`) a soma dos claros é zero e $S_d(b)$ é exatamente a soma do keystream sobre o cubo. Com $N = 1024$ um dispositivo dá um cubo de dimensão 10 e, para $d < 10$, 2^{10-d} cubos disjuntos de dimensão d (índices em blocos consecutivos de 2^d).

Hipótese criptográfica. O grau algébrico do keystream nas variáveis do nonce após r rodadas é no máximo 2^r no Ascon (S-box de grau 2) e cresce devagar por clock no Grain; quando o grau é menor que d , $S_d(b) = 0$ para toda chave (zero-sum). Antes do zero-sum exato, $S_d(b)$ tende a ser enviesada para 0. Então o teste tem dois níveis: zero-sum exato (fração de cubos com soma 0 igual a 1,0) e viés (fração significativamente acima de 0,5).

Teste. Para cada (algoritmo, rodada, d , posição limpa b): fração $f_d(b)$ de cubos de teste com $S_d(b) = 0$, sobre todos os dispositivos de teste; sob H_0 , cada soma é Bernoulli(1/2) e $f \sim \text{Binomial}$. Estatística por configuração = qui-quadrado somado sobre as posições limpas, p-valor para o BH-FDR. Reportar também o d mínimo que dá zero-sum exato por rodada, que é o resultado com cara de teorema empírico, e a figura $f_d(b) \times d$ por rodada. Para $d = 1$ o teste coincide com o Detector 1 do sub-exp. A sobre pares $(c, c+1)$; isso é a checagem de consistência entre A e B e deve bater.

Alinhamento no contador sorteado. Para o cenário realista em que o contador não começa em zero, alinhar o offset a múltiplo de 2^D (o atacante espera o contador cruzar um múltiplo; no fluxo real isso acontece a cada 2^D mensagens). Rodar `zero` e `sorteado`; a diferença entre os dois mede o efeito da vizinhança, como no v2.

Por algoritmo. Grain-128AEAD é o alvo principal (varredura densa de clocks até 128, $D = 10$). Ascon com $D = 10$ nas rodadas 3, 4, 5 e 6. Schwaemm com $D = 10$ nas rodadas 2 a 5. GIFT-COFB com $D = 8$, rodadas 2 a 6, só para registrar que o cubo passa por duas cifragens e tende a não ajudar; se ajudar, é resultado.

Bits não limpos. Para posições com `p_cubo[d][b]` entre 0,6 e 0,999, a soma dos claros atenua o sinal por $(2p - 1)$; incluir num segundo passe só se as posições limpas forem poucas (< 8) no bloco 1.

Saída. JSONL no mesmo esquema do sub-exp. A, com colunas `d`, `n_cubos`, `frac_zero`, `zero_sum_exato` (booleano), e um `.csv` com `f_d(b)` por posição para as figuras. O relatório final destaca, por algoritmo, o maior número de rodadas com detecção em qualquer `d`, lado a lado com o piso por pares.

Sub-experimento C: posição da diferença e colocação do contador

A pergunta é se a diferença de 1 bit sobrevive a mais rodadas quando entra em outra posição do nonce, ainda sem escolher nada: um contador produz pares $(n, n + 2^j)$ com diferença exatamente no bit j sempre que n é múltiplo de $2^{(j+1)}$.

Pares por posição. Dos mesmos dispositivos de $N = 1024$ do sub-exp. B, extrair, para $j = 0 \dots 9$, todos os pares $(n, n + 2^j)$ com n múltiplo de $2^{(j+1)}$; há 512 pares para cada j por dispositivo. O XOR do par passa pelos Detectores 1 e 2 do sub-exp. A, e o resultado é o piso em função de j , por algoritmo. Como os pares de j diferentes vêm das mesmas mensagens, as configurações não são independentes; o BH-FDR cobre isso de forma conservadora e o relatório deve dizê-lo.

Colocação do contador. O v2 fixa o contador big-endian no fim do nonce. Acrescentar duas colocações: little-endian (bits baixos no primeiro byte) e, no Ascon, contador na palavra `x3` em vez de `x4` (primeiros 8 bytes do nonce). Para cada colocação, rodar só $j = 0$ nas rodadas piso, piso+1 e piso+2. Isso mede se a Tabela 1 depende de onde o fabricante pôs o contador, que é uma pergunta de engenharia, não de criptanálise.

Interpretação esperada. No GIFT-128 a difusão é por permutação de bits e S-box de 4 bits: a posição decide quantas S-boxes a diferença ativa por rodada, e deve haver j com piso maior. No Ascon, `x3` e `x4` entram na S-box de 5 bits em papéis diferentes; a palavra importa. No Grain, o byte do nonce decide em que posição do LFSR a diferença entra e quantos clocks até ela chegar à função de saída; posições altas do LFSR podem sobreviver a mais clocks. No Schwaemm, a diferença cai numa das palavras do estado do SPARKLE e a posição decide se passa por uma ou duas adições modulares antes do primeiro bloco.

Saída. Mesmo esquema JSONL, com colunas `j` e `colocacao`; figura `piso × j` por algoritmo.

Sub-experimento D (opcional): GIFT-COFB com AD e rodadas de exposição

A pergunta é se o piso do GIFT-COFB é baixo porque cada chamada do bloco custa rodadas de exposição, o que explicaria a Tabela 1 e corrigiria a comparação por fração.

Rodadas de exposição. Acrescentar a `floor_algos.py`, sem alterar nada existente, uma função `exposicao(algo, rounds, n_ad_blocks)` que devolve quantas rodadas a diferença do nonce percorre até o primeiro byte observável: $\text{Ascon} = \text{pa}$; $\text{GIFT-COFB} = \text{rounds} \times (2 + \text{n_ad_blocks})$ (cifragem do nonce, mais uma por bloco de AD, mais a do primeiro bloco de mensagem — conferir no `gift_cofb_encrypt` da reimplementação se com AD vazio há uma ou duas chamadas antes de C1 e documentar); Grain = clocks de inicialização; Schwaemm = `big`. Relatar essa coluna ao lado de "rodadas" e "fração da spec" em todas as tabelas do v3.

AD. Gerar dispositivos com AD de 0, 1 e 2 blocos de 16 bytes (AD constante por dispositivo, com `label="v3-ad"`) e rodar o sub-exp. A nas rodadas 1 a 6 do GIFT. Hipótese: o piso cai uma rodada por bloco de AD. Se cair, a Tabela 1 do GIFT passa a ser lida em rodadas de exposição e o piso "3 de 40" vira "6 de 80 na exposição", consistente com os distinguidores neurais de 7 a 8 rodadas sobre uma cifragem só.

Corte pelas últimas rodadas. `PureGiftCOFB` reduz tomando as primeiras rodadas. Acrescentar a opção `corte="fim"` (últimas rodadas, com as subchaves correspondentes) e repetir as rodadas 1 a 6 em $j = 0$. Se o piso mudar, o key schedule participa do resultado e isso deve constar.

Saída. Mesmo esquema, colunas `n_ad_blocks`, `exposicao`, `corte`.

Critério de detecção, controles obrigatórios e relatório

Uma configuração (algoritmo, rodada, braço, contador, região, detector, parâmetro) conta como detectada quando passa nos quatro filtros abaixo, calculados por `exp_contador/relatorio.py` sobre o JSONL consolidado, reaproveitando `apply_bh_fdr` e a lógica de `report_floor.py`:

1. Significância sob BH-FDR com $q = 0,05$ sobre o conjunto de todas as configurações do v3 (A, B, C e D juntos). O número de testes entra no relatório.
2. Estatística acima do teto de controle: o maior valor observado nas rodadas de especificação do mesmo algoritmo e detector.
3. Monotonicidade: toda rodada abaixo também detectada no mesmo detector e parâmetro.
4. Braço `aleatorio` da mesma configuração não detectado. Uma detecção no braço aleatório invalida o detector naquela configuração e vai para a tabela de falsos positivos, não para a nota de rodapé.

Split: dispositivos de treino e teste disjuntos por chave (240/60 por lote de 300, 2 400/600 por lote de 3 000), com a seed de split fixa; viéses, limiares e modelos estimados só no treino; p-valores só no teste.

Relatório final em `docs/exp_contador.md`, gerado pelo script, com estas tabelas e figuras, nesta ordem: a Tabela 1 do v2 reproduzida pelo Detector 1 (checagem de que o v3 recupera os pisos 3/3/28/2 em $d = 1, j = 0$); piso por algoritmo e detector (pares $\times$ Detector 1, agregação $\times k$, cubos $\times d$, posição $\times j$), cada célula com rodadas, fração da spec e rodadas de exposição; k necessário para 95% por rodada; d mínimo com zero-sum exato por rodada; z_b por posição de bit no bloco 1 (figura de importância); a tabela de hipóteses H1–H6 com o veredito; e a lista de falsos positivos no braço aleatório, mesmo que vazia.

Cada linha do JSONL carrega `git_sha`, `seed_gen`, `impressao_digital` do cache de origem e `tempo_s`, para que qualquer célula seja regenerável.

Testes automatizados e critérios de aceite

Testes em `tests/test_exp_contador_*.py`, no estilo dos existentes, rodando em menos de 2 minutos com variantes pequenas (10 dispositivos, $N = 16$):

- ☐ Gerador: nonces de um dispositivo são consecutivos, o primeiro é múltiplo de 2^D , e o mesmo dispositivo cifrado duas vezes dá bytes idênticos (determinismo por seed).
- ☐ Gerador: `impressao_digital` muda quando qualquer parâmetro muda e `conferir_cache` rejeita um cache de outra configuração.
- ☐ Bits limpos: com claros ASCII sintéticos, `p_xor` e `p_cubo` marcam o bit 7 de cada byte como limpo e nenhum outro; com claros aleatórios, nenhum bit é limpo.
- ☐ Detector 1 sob H_0 : com criptogramas substituídos por bytes uniformes, a distribuição dos p -valores em 200 réplicas é uniforme (teste KS, $p > 0,01$) e a taxa de falsos positivos a $q = 0,05$ fica abaixo de 7%.
- ☐ Detector 1 com sinal plantado: ao forçar viés de 5% num bit, o p -valor global cai abaixo de $1e-6$ com 20 000 pares.
- ☐ Cubos: para um cifrador de brinquedo de grau conhecido nas variáveis do nonce (polinômio de grau 3 sobre os bits baixos), a soma é zero para todo $d \geq 4$ e enviesada para $d = 3$; e para $d = 1$ a estatística coincide com a do Detector 1 sobre pares $(c, c+1)$.
- ☐ Posição j : os pares extraídos para j têm diferença de nonce igual a exatamente 2^j , e cada mensagem aparece em no máximo um par por j .
- ☐ Exposição: `exposicao("gift", 3, 0)` devolve o valor documentado após a conferência do número de chamadas antes de C_1 , e `exposicao("ascon", r, 0) == r`.
- ☐ Relatório: um JSONL sintético com uma configuração detectada em $r = 1..3$ e não em $r = 4$ produz piso 3; o mesmo com $r = 2$ ausente produz piso descartado por não monotonicidade.

Aceite do experimento (não do código):

- ☐ O Detector 1 em $d = 1, j = 0$, contador `zero`, reproduz os pisos 3 (Ascon), 3 (GIFT), 28 (Grain) e 2 (Schwaemm) ou os supera; se ficar abaixo em algum, o v3 está errado e para aí.
- ☐ Zero detecções no braço `aleatorio` em todas as configurações, e zero nas rodadas de especificação.
- ☐ Cada hipótese H1-H6 recebe um veredito com o número que o sustenta.
- ☐ Os pisos novos aparecem com rodadas, fração da spec e rodadas de exposição.

Ordem de execução, entregas e custo estimado

Sete entregas, cada uma um commit na branch `exp-contador-v3` saída de `piso-rodadas-v2`, com testes passando antes de seguir. A primeira fase decide se o resto vale a pena.

1. **Leitura e confirmação (sem código).** Ler `run_floor.py`, `floor_algos.py`, `gift_pure_cipher.py`, `report_floor.py` e `docs/piso_rodadas.md`; confirmar os três fatos da seção de reaproveitamento e o número de chamadas do bloco antes de C1 no GIFT-COFB; escrever as respostas em `docs/exp_contador.md` antes de qualquer outra coisa.
2. **Gerador e caches** (`gerar.py` + testes). Lotes: 300 dispositivos, $N = 1024$, contador `zero`, braços `texto` e `aleatorio`, quatro algoritmos, varredura de rodadas da seção de dados. Custo: minutos para os compilados, cerca de 1 h para o GIFT puro com $N = 256$.
3. **Sub-experimento A, Detectores 1 e 2, $d = 1$** (`detectores.py`, `exp_a.py`). Primeiro a reprodução da Tabela 1; depois piso+1 e piso+2 com o lote de 3 000 dispositivos ($N = 100$ basta aqui). É o ponto de decisão: se H1 cair nos quatro algoritmos, B e C continuam, mas as expectativas de ganho caem e o relatório deve dizer isso.
4. **Sub-experimento B** (`exp_b.py`). Grain primeiro, depois Ascon, Schwaemm, GIFT. Reaproveita os caches de 2.
5. **Sub-experimento C** (`exp_c.py`). Reaproveita os caches de 2 para j ; gera os lotes de colocação do contador (pequenos: 3 rodadas por algoritmo).
6. **Sub-experimento D** (`exp_d.py`), só GIFT. Depende da confirmação de 1.
7. **Relatório** (`relatorio.py` → `docs/exp_contador.md`), com a tabela de hipóteses e as figuras.

Tempo de máquina total estimado: abaixo de 6 h num desktop, dominado pelo GIFT em Python e pelo lote de 3 000 dispositivos; o restante é desenvolvimento. Paralelizar a geração por algoritmo com processos separados, nunca threads, por causa do GIL no GIFT puro.

Decisões em aberto

O Claude Code deve perguntar antes de executar, não assumir:

- ☐ Corpus: só inglês (Gutenberg, como no v2) ou os três (inglês, português, chinês) com a máscara de bits limpos estimada por corpus? O custo triplica a geração; o ganho é a dependência do claro quantificada.
- ☐ Lote de 3 000 dispositivos na família A: rodar para os quatro algoritmos ou só para Ascon e Schwaemm, onde H1 tem mais chance?
- ☐ Sub-experimento D: entra neste PR ou fica para depois da fase 3?
- ☐ Paridades (extensão de A): executar sempre ou só se o Detector 1 falhar na rodada 4 do Ascon, como está escrito?
- ☐ Colocação do contador na palavra x3 do Ascon exige mudar a montagem do nonce no gerador; confirmar que a ordem de bytes do nonce na implementação `ascon128av13` é a esperada antes de interpretar o resultado.
- ☐ A troca de `ascon128av13` por Ascon-AEAD128 (SP 800-232) fica fora deste PR, mas o relatório deve chamar o algoritmo pelo nome correto da implementação usada. Confirmar o nome a usar no texto.